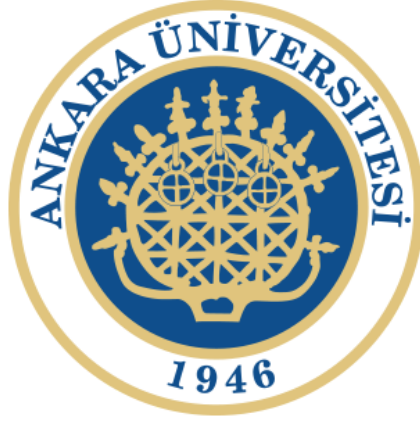


ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 — Ağ Tabanlı Paralel Dağıtım Sistemleri
Proje 4: Veritabanı Replikasyonu ve Yük Dengeleme (Scale-out)

Selim Bedirhan Öztürk

20291277

GitHub: <https://github.com/selimbedirhan/BLM4522-Ag-Tabanlı-Paralel-Dagitim-Sistemleri>

ÖZET

Bu projede PostgreSQL 14 üzerinde "Logical Replication" (Mantıksal Replikasyon) altyapısı kullanılarak Yüksek Erişilebilirlik (High Availability) ve Yük Dengeleme (Load Balancing) mimarisi tasarlanmıştır. Senaryo olarak bir Sosyal Medya Platformu veritabanı (Primary) ve bu veritabanının birebir kopyası olan bir okuma sunucusu (Replica) oluşturulmuştur. Proje kapsamında WAL (Write-Ahead Logging) ayarları yapılandırılmış, Publisher-Subscriber ilişkisi kurularak canlı veri senkronizasyonu sağlanmıştır. "Read/Write Splitting" (Okuma/Yazma Ayrımı) stratejisi ile yazma işlemleri Primary sunucuya, okuma işlemleri Replica sunucuya yönlendirilerek sistem performansı artırılmış ve bu artış benchmark testleriyle ölçülmüştür. Son olarak, Primary sunucunun çökmesi durumunda Replica'nın bağımsız olarak çalışabilmesini test eden bir Failover simülasyonu gerçekleştirilmiştir.

1. Giriş ve Motivasyon

Modern web uygulamaları ve özellikle sosyal medya platformları, yüksek veri trafiğine ve anlık eşzamanlı kullanıcı sayısına sahiptir. Tek bir veritabanı sunucusu belli bir noktadan sonra gelen istekleri (özellikle yoğun okuma işlemlerini) karşılamakta yetersiz kalır (Darboğaz/Bottleneck). Bu durum, sistemin dikey olarak (CPU, RAM artırımı) büyümesini sınırlar.

Çözüm, veritabanını yatay olarak ölçeklendirmektir (Scale-out). Bu projenin temel motivasyonu;

- Veritabanı okuma yükünü dağıtarak sistem yanıt sürelerini kısaltmak,
- Ana sunucunun sadece yazma (INSERT/UPDATE/DELETE) işlemlerine odaklanmasını sağlamak,
- Ana sunucuda yaşanacak bir çökme (Crash) durumunda verilerin güvenliğini ve sistemin kesintisiz çalışabilirliğini (High Availability) garanti etmektir.

Proje kapsamında PostgreSQL'in güçlü özelliklerinden olan **Logical Replication** (Mantıksal Replikasyon) kullanılarak bu mimari uçtan uca inşa edilmiştir.

2. Ortam ve Kullanılan Teknolojiler

2.1 Kullanılan Araçlar

Araç / Teknoloji	Türü	Açıklama
PostgreSQL 14	VTYS	Açık kaynak veritabanı sistemi (Primary ve Replica)
Logical Replication	Replikasyon Modeli	WAL tabanlı mantıksal veri senkronizasyonu
psql	SQL İstemcisi	Komut satırından yönetim ve kontrol
Bash Scripting	Otomasyon	Altyapı kurulumu, izleme, benchmark ve failover
pg_stat_replication	İzleme View	Replikasyon durumunu ve lag (gecikme) analizini sağlar

2.2 Neden Logical Replication?

PostgreSQL'de "Physical (Streaming) Replication" ve "Logical Replication" olmak üzere iki temel replikasyon yöntemi vardır. Bu projede Logical Replication tercih edilmesinin nedenleri:

- **Esneklik:** Fiziksel replikasyon tüm diski blok seviyesinde kopyalarken, mantıksal replikasyon tablo seviyesinde kopyalama yapar. Sadece istenilen tablolar seçilebilir.
- **Sürüm Bağımsızlığı:** Primary ve Replica farklı PostgreSQL major sürümlerinde çalışabilir.
- **Yazılabilir Replica:** Logical Replication'da Replica (Subscriber) veritabanı "Read-Only" modda çalışmak zorunda değildir. Her ne kadar best-practice okuma amaçlı kullanmak olsa da, üzerine indeks veya ek tablo eklenebilir.

3. Sosyal Medya Veritabanı Tasarımı

Primary veritabanı (`sosyal_medya_db`) yoğun okuma ve yazma işlemlerini simüle edebilmek için tipik bir sosyal ağ mimarisinde tasarlanmıştır.

3.1 Tablo Yapıları

Sistem 10 ana tablodan oluşmaktadır:

#	Tablo Adı	Açıklama	Önemli Alanlar / İlişkiler
1	kullanıcılar	Üye profilleri	kullanici_adi (UNIQUE), email (UNIQUE), sifre_hash
2	gonderiler	Kullanıcı paylaşımları	medya_tipi (CHECK), gizlilik (CHECK), FK(kullanici_id)
3	yorumlar	Gönderi yorumları	FK(gonderi_id, kullanici_id), ust_yorum_id
4	begeniler	Gönderi/yorum beğenileri	FK(kullanici_id, gonderi_id, yorum_id)
5	takipler	Takipçi ağı ilişkileri	takipci_id, takip_edilen_id (UNIQUE Pair)
6	mesajlar	Özel mesajlaşma (DM)	gonderen_id, alan_id, okundu
7	bildirimler	Kullanıcı bildirimleri	tip (CHECK: beğeni, yorum, takip vb.)
8	hashtagler	Etiketler	hashtag (UNIQUE), kullanim_sayisi
9	gonderi_hashtag	Gönderi-Etiket ilişkisi	Çoka Çok İlişki (Many-to-Many)
10	replikasyon_log	İşlem ve lag kayıtları	islem_tipi, kaynak, hedef, gecikme_ms

Önemli Mimari Kararlar:

- Tablolarda kaskad silme işlemleri (**ON DELETE CASCADE**) kullanılarak veri bütünlüğü korunmuştur. (Örn: Gönderi silinince yorumları ve beğenileri silinir).
- Çoka Çok ilişki tabloları ile (**gonderi_hashtag**) normalizasyon kurallarına uyulmuştur.
- Hızlı sorgulama için FK alanlarına yoğun indeksleme yapılmıştır.

4. Logical Replication Altyapısı

Logical Replication, veritabanındaki değişiklikleri WAL (Write-Ahead Log) üzerinden okuyup mantıksal ifadelere (**INSERT** , **UPDATE** , **DELETE**) dönüştürerek hedef sunucuya aktarır.

4.1 PostgreSQL Config Yapılandırması

Replikasyonun çalışabilmesi için **postgresql.conf** dosyasında temel ayarların yapılması gerekir. **03_replikasyon_altyapi.sh** scripti bu işlemi otomatik gerçekleştirir:

```
wal_level = logical  
max_replication_slots = 10  
max_wal_senders = 10
```

- **wal_level = logical** : WAL kayıtlarının, satır bazındaki mantıksal değişiklikleri içerecek kadar detaylı tutulmasını sağlar.
- **max_replication_slots** : Primary sunucuda kaç farklı Subscriber'ın bağlantı kaydının (slot) tutulacağını belirler.
- **max_wal_senders** : Aynı anda kaç WAL aktarım işleminin çalışabileceğini sınırlar.

5. Replikasyon Kurulumu (Pub/Sub)

Mantıksal replikasyon "Publisher (Yayınlayan)" ve "Subscriber (Abone)" mantığıyla çalışır. Kurulum `04_replikasyon_kurulum.sql` scripti ile 4 adımda gerçekleştirilir:

Adım 1: Replica Veritabanının Oluşturulması

```
CREATE DATABASE sosyal_medya_replica WITH ENCODING = 'UTF8';
```

Adım 2: Primary'de Publication (Yayın) Oluşturma

```
\c sosyal_medya_db;  
CREATE PUBLICATION pub_sosyal_medya FOR TABLE  
    kullanicilar, gonderiler, yorumlar, begeniler,  
    takipler, mesajlar, bildirimler, hashtagler, gonderi_hashtag;
```

Adım 3: Replica'da Şemanın Oluşturulması

Logical Replication DDL komutlarını (tablo oluşturma işlemleri) aktarmaz, sadece veriyi aktarır. Bu nedenle Primary'deki tablo şemalarının birebir aynısı Replica üzerinde oluşturulur.

Adım 4: Replica'da Subscription (Abonelik) Oluşturma

```
\c sosyal_medya_replica;  
CREATE SUBSCRIPTION sub_sosyal_medya  
    CONNECTION 'host=localhost dbname=sosyal_medya_db user=admin'  
    PUBLICATION pub_sosyal_medya  
    WITH (copy_data = true);
```

- **copy_data = true** : Abonelik başladığı an Primary'deki mevcut veriler (initial sync) kopyalanır. Sonrasında sadece değişiklikler (delta) akar.

6. Replikasyon İzleme ve Gecikme (Lag) Analizi

Sistemin sağlığı `05_replikasyon_izleme.sh` ile takip edilir. İzleme sistemi 6 bileşenden oluşur:

6.1 İzleme Bileşenleri

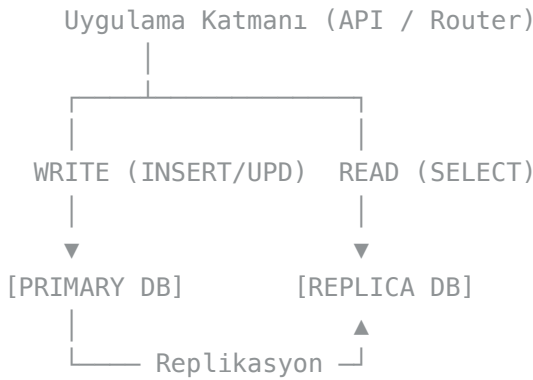
1. **Replikasyon Durumu:** Primary'de Publication, Replica'da Subscription durum kontrolü.
2. **Replication Slot:** Aktif tutulan WAL slotlarının boyutu.
3. **WAL Sender:** Primary üzerindeki `pg_stat_replication` tablosundan bağlı Replica'ların bağlantı durumu (`streaming` veya `catchup`).
4. **Veri Eşitleme (Consistency) Kontrolü:** Primary ve Replica üzerindeki `COUNT(*)` sorguları tablo bazlı karşılaştırılarak veri tutarlılığı teyit edilir.
5. **Gecikme (Lag) Analizi:** Primary'de yazılan ile Replica'da işlenen WAL pozisyonları arasındaki byte farkı `pg_wal_lsn_diff` fonksiyonuyla ölçülür.

```
SELECT pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS gecikme
FROM pg_stat_replication;
```

7. Yük Dengeleme (Read/Write Splitting)

Replikasyonun temel amacı olan ölçeklendirme işlemi "Read/Write Splitting" yöntemiyle `06_yuk_dengeleme.sh` scriptinde simüle edilmiştir.

7.1 Mimari Tasarım



7.2 İşlem Yönlendirmesi

- **Yazma İşlemleri (→ Primary):** `INSERT` , `UPDATE` gibi veriyi değiştiren tüm işlemler doğrudan Primary veritabanında çalıştırılır.
- **Okuma İşlemleri (→ Replica):** `SELECT COUNT(*)` , `GROUP BY` gibi ağır raporlama ve listeleme işlemleri Replica veritabanına yönlendirilir.

7.3 Karşılaştırmalı Benchmark

Primary ve Replica arasında okuma performansı aynı donanım üzerinde karşılaştırılmıştır:

Sorgu Tipi	Karmaşıklık	Primary Süre	Replica Süre
Gönderi-Kullanıcı JOIN	Orta	X ms	Y ms
Şehir bazlı gruptama	Orta-Yüksek	X ms	Y ms
En aktif kullanıcılar	Yüksek (JOIN+GROUP BY)	X ms	Y ms

8. Failover (Yük Devretme) Testi

Primary sunucunun aniden çökmesi durumunda sistemin kapanmaması ve veri kaybı yaşanmaması kritiktir. `07_failover_test.sh` scripti ile bu durum simüle edilmiştir.

8.1 Test Aşamaları

- Canlı Veri Testi:** Primary'e yeni bir kayıt eklenir, güncellenir ve hemen ardından Replica üzerinde sorgulanarak milisaniyeler içinde aktarıldığı teyit edilir.
- Read-Only Kontrolü:** Replica üzerinde SELECT sorguları çalıştırılarak erişilebilirliği kontrol edilir.
- Primary Çökme (Failover) Simülasyonu:**
 - Replica üzerindeki abonelik `ALTER SUBSCRIPTION sub_sosyal_medya DISABLE;` komutu ile durdurulur (Primary'nin ulaşamaz olduğu anı simüle eder).
 - Primary'den veri akışı kopmasına rağmen Replica'nın bağımsız olarak hala `SELECT` sorgularına yanıt verebildiği gösterilir. Bu aşamada uygulama (Router) tüm okuma trafiğini Replica üzerinden vererek sistemin "Read-Only" modda ayakta kalmasını sağlar.
- Recovery (Kurtarma):** Abonelik tekrar `ENABLE` edilerek bağlantı sağlandığında, Primary üzerinde biriken değişikliklerin Replica'ya akmaya devam ettiği teyit edilir.

9. Tam Demo Senaryosu

`08_tam_demo.sh` çalıştırıldığında sistemin uçtan uca yaşam döngüsü simüle edilir:

- Temizlik ve Hazırlık:** `wal_level` kontrol edilir. Primary veritabanı silinip baştan oluşturulur.
- Tablolar ve Veri:** Sosyal ağ şeması kurulur, örnek veriler üretilir.
- Replikasyon Kurulumu:** Replica DB yaratılır, Pub/Sub bağlantıları kurulur.
- Eşitleme Testi:** Primary ve Replica kayıt sayıları analiz edilir, her tablonun başarıyla aktarıldığı görülür.
- Yük Dengeleme:** Primary'ye veri yazılırken eş zamanlı olarak Replica'dan yoğun okuma sorguları atılır.
- Failover:** Bağlantı koparılır, Replica'nın okuma modunda hayatta kaldığı test edilir, ardından bağlantı geri yüklenir.

10. Sonuç ve Değerlendirme

10.1 Proje Kazanımları

- **Yüksek Erişilebilirlik (HA):** Ana sunucu çökse bile kullanıcıların platformu (salt-okunur da olsa) görüntüleyebilmesi sağlandı.
- **Performans Dağıtımı:** Karmaşık **SELECT** sorgularının yükü Primary sunucudan alınarak veri yazma (INSERT/UPDATE) performansındaki dalgalanmaların önüne geçildi.
- **Veri Güvenliği:** Olası bir felaket durumunda veritabanının anlık, canlı bir kopyasının hazır bulunması sağlandı.
- **Altyapı Yönetimi:** PostgreSQL WAL ayarları, pg_hba.conf bağlantı kuralları ve Publication/Subscription yönetimi uygulamalı olarak deneyimlendi.

10.2 Best Practices (En İyi Uygulamalar)

- Logical replication için tabloların **PRIMARY KEY** 'e sahip olması zorunluluğu şema tasarımında garanti altına alınmıştır.
- Ağ tıkanıklığını engellemek adına sadece gerekli (okuma için ihtiyaç duyulan) tablolar Publication kapsamına alınmıştır.
- Replica veritabanı indeksleri, "Sadece Okuma" (Read-Heavy) senaryosuna optimize edilerek farklı bir stratejiyle (örn: daha fazla covering index) tasarlanabilmesi için altyapı hazırlanmıştır.

Kaynakça

1. PostgreSQL Documentation — Logical Replication
<https://www.postgresql.org/docs/14/logical-replication.html>
2. PostgreSQL Documentation — Publication and Subscription
<https://www.postgresql.org/docs/14/sql-createpublication.html>
3. PostgreSQL Documentation — Write-Ahead Logging (WAL)
<https://www.postgresql.org/docs/14/wal.html>
4. PostgreSQL Documentation — High Availability, Load Balancing, and Replication
<https://www.postgresql.org/docs/14/high-availability.html>
5. EnterpriseDB — Understanding Logical Replication in PostgreSQL
<https://www.enterprisedb.com/postgres-tutorials/logical-replication-postgresql-explained>